POC Gestos-Mouse

Control del mouse del sistema operativo con gestos de manos, y una capa vectorial de afordancia ("segundas manos") sobre el escritorio.

Estado	Concepto + arquitectura inicial. Listo para andamiaje en IDE agéntico.
Origen	POC **3d-cube-parallax** (repo vandalit/metamedia): MediaPipe + Three.js en navegador.
Objetivo del POC	Validar el pipeline cámara -> landmarks -> gestos -> (cursor + capa vectorial) y la *sensación* de control.
Plataforma objetivo	Windows nativo (no WSL: sin acceso a cámara ni mouse del host).

1. Visión y distinciones de diseño

- El usuario mueve las manos frente a la cámara; el **índice controla el cursor** y el gesto **push** (un dedo extendido, resto recogido, mano que se acerca) ejecuta el click. Después: pinch, grab y gestos de manipulación de interfaz.
- **La capa de manos es un componente vectorial de afordancia** (máscaras semitransparentes de manos, dedos y gestos). No es un viewport ni un canvas de la app: es una representación que el usuario reconoce como "sus segundas manos". La ventana del SO es solo el portador, invisible para el usuario.
- **Desvanecimiento progresivo:** las manos se simplifican a medida que el usuario gana familiaridad (mano completa, esqueleto, puntas de dedos, solo cursor), y reaparecen cuando el gesto es ambiguo.
- **Modo no invasivo:** la app vive en la bandeja; se activa/desactiva con atajo global y con gestos maestros.
- El UX es el terreno de juego: umbrales, tiempos y estética deben ser **parámetros ajustables en vivo**, no constantes enterradas en el código.

2. Alcance del POC

Dentro	Fuera (por ahora)
Una mano, una pantalla, Windows 10/11. Índice -> cursor, push -> click izquierdo, clutch (palma abierta = pausa). Capa vectorial con mano y fade manual. Tray + atajo global.	Multimonitor, DPI mixto, juegos en pantalla completa exclusiva, ventanas elevadas (UAC), macOS/Linux, dos manos, calibración automática, métrica de familiaridad real (en el POC el fade es manual).

3. Arquitectura

Tres piezas desacopladas. Se puede cambiar el host (Electron por Tauri, por ejemplo) sin tocar la lógica de gestos.

El bus de eventos tiene **dos consumidores** independientes: el **driver de cursor** (inyecta mouse al SO) y el **renderer vectorial** (dibuja la capa de manos). Ambos reciben los mismos eventos.

Procesos de Electron

Proceso	Responsabilidad
Main	Ciclo de vida, Tray, `globalShortcut`, creación de ventanas, driver de cursor (módulo nativo), bus de eventos hacia los renderers.

Proceso	Responsabilidad
Vision (ventana oculta)	Captura de cámara + MediaPipe HandLandmarker + filtrado + FSM. `backgroundThrottling: false` para que no se frene al estar oculta o minimizada.
Overlay (ventana transparente)	Sin bordes, `transparent`, `alwaysOnTop`, `setIgnoreMouseEvents(true)` (click-through), no focusable. Renderiza SVG/Canvas 2D de las manos.
Panel de ajustes (opcional)	Sliders de parámetros en vivo (sección 9). Puede ser una ventana normal abierta desde la bandeja.

4. Stack propuesto

Capa	Tecnología	Nota
Host	Electron + TypeScript	Tauri queda como migración futura si el peso importa.
Visión	MediaPipe Tasks Vision: HandLandmarker (WASM/GPU)	Mismo paquete que el POC del cubo. Modelo local, sin red.
Capa vectorial	SVG (o Canvas 2D); paths por dedo	SVG facilita morphing y fades; migrar a Canvas si hay que optimizar.
Mouse del SO	Librería nativa (nut.js, robotjs) o llamadas directas a SendInput	**Verificar cuál está mantenida y compila con la versión de Electron elegida antes de comprometerse.**
Control	Tray, globalShortcut	Atajo siempre disponible como salvavidas.
Pruebas	Vitest para la FSM y los filtros	La FSM debe poder probarse con grabaciones de landmarks, sin cámara.

5. Estructura de carpetas sugerida

```
gesture-mouse/
  package.json  tsconfig.json  electron.vite.config.ts
  src/
    main/            index.ts  tray.ts  shortcuts.ts  windows.ts
                     cursor-driver.ts      # inyección de mouse (interfaz + impl nativa)
    vision/          index.html  camera.ts  hand-tracker.ts  one-euro.ts
    gestures/        fsm.ts  features.ts  calibration.ts  types.ts   # sin Electron
    overlay/         index.html  hand-renderer.ts  affordance-fade.ts  hand-skeleton.svg
    shared/          events.ts  config.ts  ipc-channels.ts
  recordings/        # secuencias de landmarks (JSON) para tests y para ajustar umbrales
  tests/             fsm.test.ts  one-euro.test.ts  push-detector.test.ts
```

6. Contratos principales

Tipos compartidos (esbozo). La FSM consume `HandFrame` y emite `GestureEvent`.

```
type Vec3 = { x: number; y: number; z: number };   // normalizadas 0..1 (MediaPipe)

interface HandFrame {
  t: number;                    // timestamp ms
  landmarks: Vec3[];            // 21 puntos, ya filtrados (One Euro)
  handedness: 'Left' | 'Right';
  handScale: number;            // distancia muneca-nudillo medio, normaliza todo
}

type GestureEvent =
  | { type: 'pointer-move'; x: number; y: number }         // pantalla, px
  | { type: 'click'; at: { x: number; y: number }; button: 'left' }
  | { type: 'drag-start' | 'drag-end'; at: { x: number; y: number } }
  | { type: 'pinch'; amount: number }                      // 0..1
  | { type: 'grab'; amount: number }                       // 0..1
  | { type: 'clutch'; engaged: boolean }
  | { type: 'master-toggle' }                              // gesto maestro on/off
  | { type: 'hand-lost' };

interface CursorDriver {
  move(x: number, y: number): void;
  down(): void;  up(): void;  click(): void;
}
```

7. Máquina de estados y algoritmos clave

Estados

```
OFF --(atajo | gesto maestro)--> IDLE
IDLE --(palma abierta sostenida)--> CLUTCH      # pausa: la mano se ve pero no controla
IDLE --(indice extendido, resto recogido)--> POINT
POINT --(escala crece rapido)--> PUSH_ARMED --(umbral superado)--> CLICK --> POINT
POINT --(pinch sostenido)--> DRAG --(suelta)--> POINT
cualquier estado --(mano perdida > N ms)--> IDLE
```

Algoritmos

- **Filtro One Euro** sobre cada landmark antes de la FSM (frecuencia de corte mínima y beta ajustables). Sin él, el cursor tiembla.
- **Mapeo cámara -> pantalla:** zona activa (rectángulo dentro del encuadre, p. ej. 60% central), espejo en X, interpolación a coordenadas de pantalla. Así se llega a los bordes sin estirar el brazo.
- **Detección de push:** la profundidad monocular es ruidosa, por lo que no se usa el valor Z directamente. Se usa la **tasa de cambio de** `handScale` (la mano crece al acercarse) combinada con la postura "índice extendido, resto recogido".
- **Compensación de desplazamiento al clickear:** al empujar, la punta del índice se mueve y el click cae en otro punto. Mantener un buffer circular de ~150-200 ms de posiciones y **emitir el click en la posición previa al inicio del push** (o congelar el cursor al armar el push).
- **Pinch y grab:** se calculan con distancias entre landmarks normalizadas por `handScale` (pinch: pulgar-índice; grab: promedio de distancias punta-palma). Usar histéresis (umbral de entrada distinto al de salida).
- **Clutch:** palma abierta sostenida congela el control. Reduce la fatiga del brazo y los falsos positivos.

8. Capa vectorial de afordancia

- Mano dibujada con paths SVG por dedo; los 21 landmarks mueven los puntos de control. Sombra/tinte semitransparente, estilo propio (a definir en la fase de UX).

- **Alineación obligatoria:** la punta del índice vectorial coincide exactamente con el hotspot del cursor real. Si no, el paralelo mental se rompe.
- **Niveles de afordancia** (`affordance-fade.ts`): L3 mano completa, L2 esqueleto, L1 puntas de dedos, L0 solo cursor. En el POC el nivel se fija a mano desde el panel; a futuro lo gobierna una métrica de familiaridad (aciertos de click, sesiones).
- **Feedback de estado:** la mano cambia sutilmente al armar el push, al hacer pinch o al entrar en clutch. Reaparece temporalmente a L3 si el gesto es ambiguo.
- La ventana del overlay nunca debe robar foco ni interceptar clicks.

9. Parámetros ajustables en vivo

Parámetro	Qué controla	Punto de partida
oneEuro.minCutoff / beta	Suavizado vs. latencia	Ajustar a ojo con la grabación de prueba
activeZone	Rectángulo útil del encuadre	~60% central
push.scaleRate	Velocidad de crecimiento de handScale que arma el push	Calibrar con grabaciones
push.bufferMs	Ventana de posición previa para el click	150-200 ms
pinch.enter / pinch.exit	Histéresis del pinch	Entrada < salida
clutch.dwellMs	Tiempo de palma abierta para pausar	~400-600 ms
master.dwellMs	Tiempo del gesto maestro (evita activación accidental)	~1000 ms
affordance.level	L0-L3	L3 al empezar

10. Fases y criterios de aceptación

Fase	Entregable	Criterio de aceptación
0. Spike (~2 h)	Electron + overlay transparente + vision oculta. Un círculo sigue la punta del índice. **Sin mover el mouse todavía.**	El círculo sigue el dedo con latencia aceptable y el overlay no bloquea clicks.
1. Cursor	One Euro, zona activa, driver de cursor, tray + atajo.	Se alcanzan los bordes de la pantalla y el atajo apaga el modo siempre.
2. Capa vectorial	Mano SVG alineada con el cursor, niveles L0-L3 manuales.	La punta vectorial coincide con el cursor real.
3. Click	Detector de push + buffer pre-click + clutch.	Click acertado sobre un blanco de ~40 px en la mayoría de intentos (medir con una página de prueba).
4. Gestos extra	Pinch/drag, grab, gestos maestros, panel de parámetros.	Arrastrar una ventana y alternar el modo con gesto.
5. Pulido	Multimonitor y DPI, fade por familiaridad, grabación/reproducción de sesiones.	A definir tras validar la sensación de control.

11. Riesgos y limitaciones conocidas

- **Ventanas elevadas / UAC:** Windows ignora la inyección de mouse hacia procesos con más privilegios, salvo que la app corra como administrador.
- **Pantalla completa exclusiva:** algunos juegos tapan el overlay.

- **Multimonitor y DPI:** requiere un overlay por pantalla y conversión con factor de escala. Se deja para después del POC.
- **Iluminación y cámara:** el tracking se degrada con poca luz o manos parcialmente fuera de encuadre.
- **Fatiga:** el brazo en el aire cansa; el clutch y la zona activa reducen el problema, pero hay que probarlo con sesiones largas.
- **Librería de mouse nativa:** puede fallar al compilar contra Electron; tener un plan B (llamada directa a SendInput vía módulo nativo propio).
- **Privacidad:** el procesamiento debe ser local; la cámara no debe enviar frames a ningún servicio. Indicador visible cuando la cámara esté activa.

12. Instrucciones de arranque para el agente

Pegar como primer prompt en el IDE agéntico:

```
Contexto: lee este brief completo. Vamos a construir un POC en Electron + TypeScript
que usa MediaPipe HandLandmarker para controlar el mouse de Windows con gestos y
dibuja una capa vectorial semitransparente de manos como overlay.

Tarea 1 (Fase 0, spike): andamiaje del proyecto segun la seccion 5, con:
  - ventana 'vision' oculta (backgroundThrottling: false) con captura de camara
    y HandLandmarker, enviando landmarks al main por IPC
  - ventana 'overlay' transparente, alwaysOnTop, click-through, no focusable,
    que dibuja un circulo en la punta del indice
  - Tray con 'Activar/Desactivar' y un globalShortcut equivalente
NO muevas el mouse del SO todavia. Deja la FSM de gestos y el cursor-driver
como interfaces vacias segun la seccion 6.

Reglas: la logica de gestos no importa nada de Electron; los parametros viven en
shared/config.ts; antes de elegir libreria de mouse nativa, comprueba que este
mantenida y compile con la version de Electron instalada, y consultame.
Documenta cada decision en docs/decisiones.md.
```